

Cahier des charges — EpiTrello

1. Résumé exécutif

EpiTrello est une application de gestion de tableaux type Trello (kanban, workspaces, cartes, collaboration). L'objectif est de livrer une plateforme moderne, scalable et prête à évoluer (temps réel, templates, automatisations), avec une base technique propre et des déploiements automatisés.

2. Objectifs

- Proposer une UX fluide pour la gestion de tableaux, listes et cartes.
- Supporter la collaboration (workspaces, membres, rôles).
- Préparer l'architecture pour le temps réel (sockets) et les templates de board.
- Standardiser le workflow de contribution (issues, PR, branches par feature).
- Assurer la qualité via CI/CD et code coverage backend.

3. Périmètre fonctionnel (actuel + cible)

Déjà en place

- Frontend Next.js (pages landing, login/signup, account settings, boards).
- Sidebar boards avec favoris/récents/workspaces.
- Pages de paramètres utilisateur (profil, activité, paramètres, accessibilité).
- Navbar commune.

Cible proche

- Auth OAuth (GitHub, Discord) + email/mdp.
- Gestion complète des workspaces (membres, rôles, paramètres, danger zone).
- Gestion des boards (création, visibilité, templates).

Évolutions prévues

- Temps réel via WebSocket (cartes, listes, commentaires).
- Templates de boards.
- Notifications et activité.
- Filtrage avancé et recherche.

4. Architecture technique

Frontend

- Next.js 14 + React 18.
- Tailwind CSS + shadcn/ui.
- Apollo Client pour GraphQL.

Backend

- Node.js + Express + Apollo Server (GraphQL).
- Prisma ORM.
- PostgreSQL.

Infrastructure

- Docker Compose (dev/prod).

- Terraform pour l'infra.
- Provider cloud : Google Cloud Platform (GCP).

5. Benchmark technologique (choix justifiés)

Frontend

- **Next.js** vs Vite+React: Next.js apporte SSR/SSG, routing intégré, et un déploiement simple. Vite est plus léger mais nécessite plus d'assemblage.

Backend

- **Apollo Server + Express** vs NestJS: Apollo/Express est plus direct et léger pour GraphQL, NestJS apporte davantage de structure mais peut être plus lourd pour ce besoin.

API

- **GraphQL** vs REST: GraphQL réduit le sur-fetching, permet de composer les vues rapidement et convient bien aux interfaces complexes (boards, cartes, membres).

Base de données

- **PostgreSQL** vs MySQL/MongoDB: PostgreSQL est robuste pour les relations (workspaces, boards, permissions) et bien supporté par Prisma.

Infra

- **GCP + Terraform** vs alternatives (AWS/Azure): choix motivé par la capacité à automatiser l'infra via Terraform et la cohérence avec les environnements de déploiement visés.

6. Environnements

- **Dev** : Docker Compose dev + variables `env.dev.template`.
- **Prod** : Docker Compose prod + variables `env.prod.template`.

7. CI/CD & Qualité

- CI/CD attendu avec pipeline de build/test/deploy.
- Intégration **Codecov** pour couverture backend.
- Déploiements automatisés via Terraform (GCP).

8. Organisation du projet (Git/GitHub)

- Branching par **feature**.
- `main` = branche de production.
- PR obligatoires, review avant merge.
- Issues et PR gérées via **GitHub CLI** (`gh`), avec assignation et labels.
- Les PR indiquent les issues clôturées (`Closes #id`).

9. Plan de livraison (macro)

1. Stabilisation auth + pages publiques.
2. Boards + workspaces (CRUD, permissions).
3. Temps réel et activité.
4. Templates et automatisations.
5. Optimisations + monitoring.

10. Risques & mitigations

- **OAuth instable** : config stricte des redirect URI, tests multi-env.
- **Temps réel** : introduire progressivement via WebSocket + feature flags.
- **Complexité UI** : composants UI standardisés (shadcn/ui).

11. Livrables

- Frontend complet (Next.js).
- Backend GraphQL + DB.
- Infra Terraform GCP.
- Documentation technique + scripts.

Document évolutif, mis à jour au fil des features et livraisons.